

SENTINAL SECURITY AUDIT REPORT

Target Scope: None | Scan Mode: STANDARD | Auth: DEV_MODE | Date: September 08, 2026 at 08:56 UTC

Security Score	DAST Coverage	Critical Flaws	High Flaws	Total Findings
85.4/100	100.0% (NOT_APPLICABLE)	2	2	7

2. Executive Summary

****Security Assessment Executive Summary****

****Overall Posture:**** CRITICAL POSTURE

The automated assessment of None identified immediate critical exposures requiring urgent remediation. A total of 7 findings were recorded, including 2 Critical and 2 High severity vulnerabilities.

****Key Risk Highlights:****

- Total Discovered Findings: 7 (Critical: 2, High: 2, Medium: 3, Low: 0)
- Correlated Exploit Chains: 0
- Primary Concern Areas: Injection, Container Security, Code Execution, Vulnerable Dependency

Leadership is advised to authorize immediate sprint allocation for remediation of critical issues and enforce automated security gating in CI/CD pipelines.

4. Prioritized Vulnerability & Threat Inventory

Severity & Risk	Engine	Vulnerability & Threat Scenario	Location & Remediation
MEDIUM Risk: 42.1/100 Blast: Information Disclosure &	SAST sentinal-sast	Dockerfile Container Running as Root Threat: The vulnerability exposes implementation details, internal software versions, or unhardened endpoints that assist attackers in reconnaissance and multi-stage attack chaining...	Dockerfile Fix: Add `USER nonroot` or a dedicated unprivileged user before the ENTRYPOINT/CMD...
HIGH Risk: 94.3/100 Blast: Database Takeover & Sens	SAST sentinal-sast	Potential NoSQL Injection Threat: An attacker can supply crafted SQL payloads to bypass authentication barriers, extract proprietary customer PII and database tables, modify financial or user records, or execute ad..	app/data/allocations-dao.js Fix: Sanitize query keys using mongo-sanitize or enforce strict schema validation...
CRITICAL Risk: 100.0/100 Blast: Full Server Takeover & R	SAST sentinal-sast	Arbitrary Code Execution via eval() or Dynamic Function Threat: An adversary can execute arbitrary operating system commands with web process privileges, allowing them to spawn reverse shells, install backdoors/ransomware, and pivot laterally i..	app/routes/contributions.js Fix: Avoid `eval()` and `new Function()`. Use structured JSON parsers or declarative data structures...
MEDIUM Risk: 54.5/100 Blast: Third-Party Component Ex	SCA osv-scanner	Vulnerable Dependency: debug (3.2.6) - GHSA-gxpj-cx7g-858c Threat: An outdated third-party dependency contains known, publicly documented CVEs with available exploit payloads, allowing automated scanners and threat actors to target known flaws in ..	package-lock.json Fix: Upgrade decode-uri-component to a secure version. Upgrade to latest version...
MEDIUM Risk: 54.5/100 Blast: Third-Party Component Ex	SCA osv-scanner	Vulnerable Dependency: body-parser (1.15.1) - GHSA-qwcr-r2fm-qrc7 Threat: An outdated third-party dependency contains known, publicly documented CVEs with available exploit payloads, allowing automated scanners and threat actors to target known flaws in ..	package.json Fix: Upgrade body-parser to a secure version. Upgrade to latest version...

HIGH Risk: 94.3/100 Blast: Cloud & Third-Party Serv	SECRETS gitleaks	Exposed Secret: Database Connection String with Password Threat: Hardcoded credentials, API keys, or private cryptographic keys exposed in code or public responses allow adversaries to authenticate directly to third-party SaaS services, cloud ac..	README.md Fix: Immediately revoke/rotate the leaked credential, remove it from git history using git-filter-repo or BFG, and store secrets in environment v..
CRITICAL Risk: 100.0/100 Blast: Cloud & Third-Party Serv	SECRETS gitleaks	Exposed Secret: Private Cryptographic Key Threat: Hardcoded credentials, API keys, or private cryptographic keys exposed in code or public responses allow adversaries to authenticate directly to third-party SaaS services, cloud ac..	artifacts/cert/server.key Fix: Immediately revoke/rotate the leaked credential, remove it from git history using git-filter-repo or BFG, and store secrets in environment v..

5. Strategic Remediation Playbook

****Step-by-Step Remediation Actions:****

- ****Code Execution:**** Avoid `eval()` and `new Function()`. Use structured JSON parsers or declarative data structures.
- ****Private Key:**** Immediately revoke/rotate the leaked credential, remove it from git history using git-filter-repo or BFG, and store secrets in environment variables or a Secret Vault.
- ****Injection:**** Sanitize query keys using mongo-sanitize or enforce strict schema validation.
- ****Database Credentials:**** Immediately revoke/rotate the leaked credential, remove it from git history using git-filter-repo or BFG, and store secrets in environment variables or a Secret Vault.
- ****Container Security:**** Add `USER nonroot` or a dedicated unprivileged user before the ENTRYPOINT/CMD.